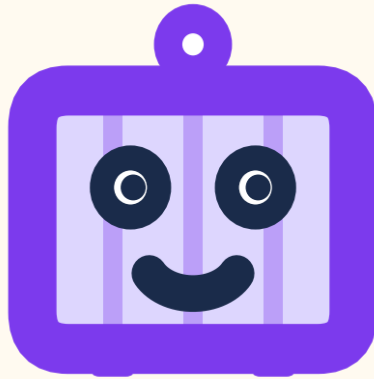


TERRAFORM

Infrastructure as Code, the illustrated field guide



with Cargo, your guide

Part of the Interview-Preparation illustrated series

Table of Contents

1. Declarative, Not Imperative
2. The Core Workflow
3. State: Terraform's Memory
4. Providers and Resources
5. terraform plan: The Diff
6. Modules: Reusable Infrastructure
7. Remote State and Locking
8. Cheat Sheet

Terraform's entire mental model boils down to a handful of ideas: describe desired state, let Terraform diff it against reality, and let it reconcile the difference. This guide builds intuition for exactly that loop.

CHAPTER 1

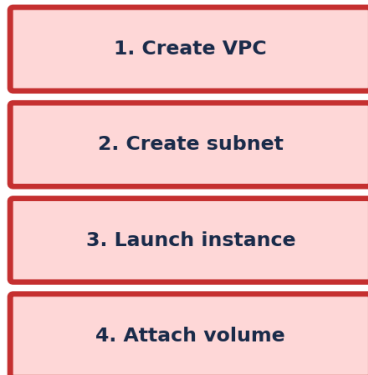
Declarative, Not Imperative

An imperative approach to infrastructure means writing a script that performs steps in order: create this, then that, then attach this to that. It works, but nothing stops the script from being run twice, half-run, or run against a system that's already partially set up in an unexpected way.

Terraform takes a declarative approach instead: you describe the end state you want -- "this VPC, this subnet, this instance should exist" -- and Terraform figures out what actions are actually needed to get from wherever things currently stand to that described state.

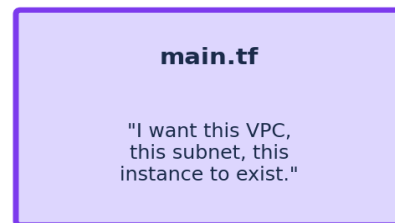
Declarative, Not Imperative

Imperative (a script)



*You script every step,
in order, by hand.*

Declarative (Terraform)



*Terraform figures out
how to get there.*

IN SHORT: You describe the destination. Terraform works out the route -- including safely handling the case where you're already partway there.

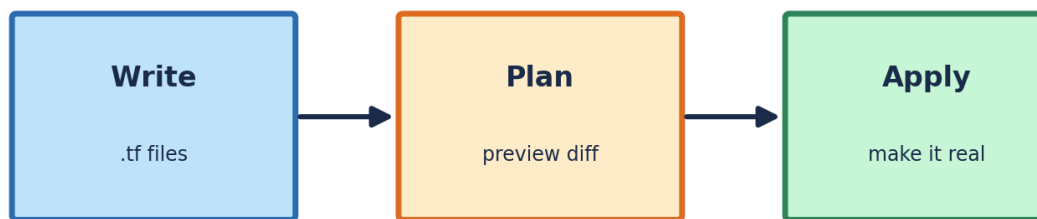
CHAPTER 2

The Core Workflow

Nearly everything in Terraform boils down to three steps, repeated: write configuration in .tf files, run terraform plan to see exactly what would change, and run terraform apply to actually make those changes.

The separation between plan and apply is deliberate -- it gives you a chance to review precisely what's about to happen (create this, modify that, destroy this other thing) before committing to it, rather than infrastructure changes happening as an opaque side effect of running a script.

The Core Workflow



IN SHORT: Write, then plan, then apply. The plan step is your safety net -- always read it before applying, especially in production.

CHAPTER 3

State: Terraform's Memory

Terraform needs to track what it has actually created, separately from your configuration files -- that's what `terraform.tfstate` is for. It maps your configuration's resources to real-world resource IDs.

A plan compares three things: your configuration (desired), the state file (what Terraform believes exists), and a fresh read of the real infrastructure (what actually exists). If someone changes something by hand in the cloud console, state and reality drift apart -- a common, genuinely confusing source of unexpected plan output.

State: Terraform's Memory



*plan compares all three. State drifting from reality
(a manual console change) is a common source of surprises.*

IN SHORT: State is Terraform's record of reality. Manual, out-of-band changes to your infrastructure are the number one cause of state drift and confusing plans.

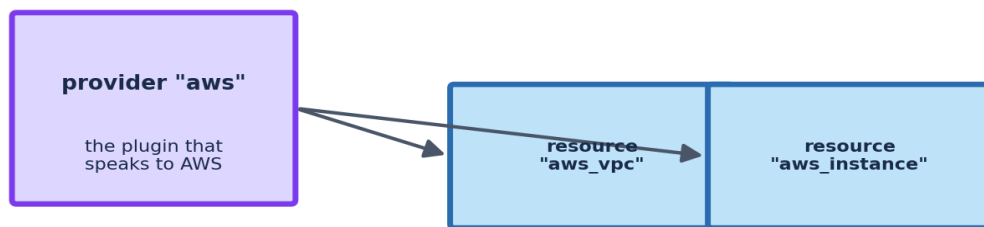
CHAPTER 4

Providers and Resources

A provider is a plugin that knows how to communicate with a specific platform's API -- AWS, GCP, Azure, and hundreds of others each have one. You declare which providers your configuration needs, and Terraform downloads and uses them.

A resource is a specific thing you want that provider to manage -- an `aws_instance`, an `aws_vpc`. Each resource block maps directly onto one real, trackable object in that platform.

Providers and Resources



*A provider knows HOW to talk to a specific platform (AWS, GCP, Azure).
A resource is WHAT you actually want that provider to create.*

IN SHORT: Providers are the how (which platform, which API). Resources are the what (the specific thing you want to exist).

CHAPTER 5

terraform plan: The Diff

Running plan produces a clear, symbol-annotated diff: a + prefix means a resource will be created, ~ means an existing resource will be modified in place, and - means a resource will be destroyed.

This preview happens entirely before anything actually changes -- giving you (or a reviewer, in a pull request) the chance to catch an unintended destroy before it happens, rather than discovering it only after real infrastructure is already gone.

terraform plan: The Diff

+ resource "aws_instance" "web"

~ instance_type: t2.micro -> t3.micro

- resource "aws_instance" "old"

*Plan shows exactly what will change BEFORE anything actually happens.
No surprises -- review the diff, then apply.*

IN SHORT: Always read the plan output carefully, especially watching for unexpected '-' (destroy) lines -- that's the single most consequential thing to catch before applying.

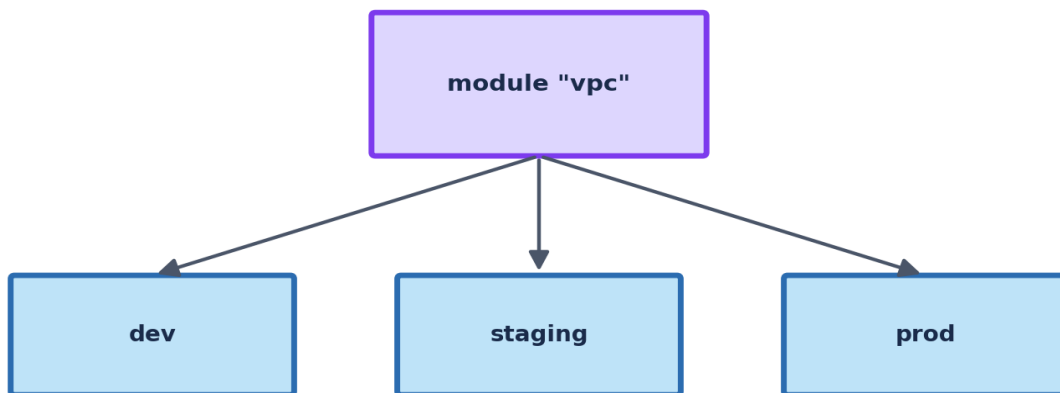
CHAPTER 6

Modules: Reusable Infrastructure

Copying and pasting the same VPC configuration across dev, staging, and production environments means every fix or update has to be applied in three separate places, and they inevitably drift apart over time.

A module packages a piece of configuration once, parameterized by input variables. Each environment then calls that same module with different inputs -- one source of truth for the actual infrastructure logic, reused everywhere it's needed.

Modules: Reusable Infrastructure



Write the VPC configuration once as a module. Reuse it, with different inputs, across every environment.

IN SHORT: Write infrastructure logic once, as a module. Reuse it with different inputs instead of copy-pasting the same configuration across environments.

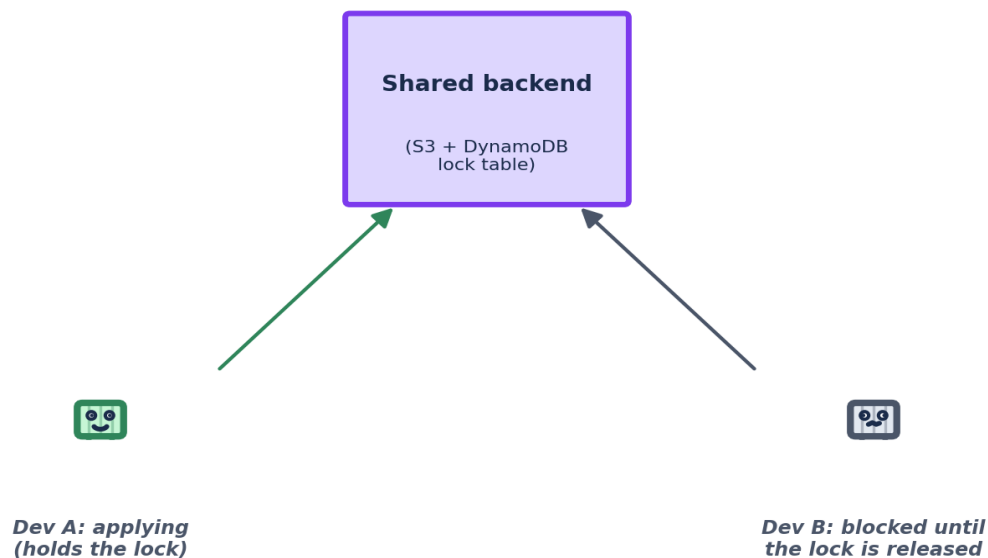
CHAPTER 7

Remote State and Locking

By default, state lives in a local file on whoever's machine ran Terraform -- unworkable the moment more than one person needs to manage the same infrastructure.

A remote backend (like an S3 bucket) stores state centrally, so every team member and CI pipeline reads and writes the same shared state. Locking (often via a DynamoDB table) prevents two applies from running concurrently against that same state, which could otherwise corrupt it or produce conflicting, unpredictable changes.

Remote State and Locking



IN SHORT: Local state doesn't survive a team. Remote state plus locking is the baseline for any Terraform setup with more than one contributor.

CHAPTER 8

Cheat Sheet

The commands and blocks you'll use constantly.

TERRAFORM CHEAT SHEET



Core Workflow

terraform init - download providers

terraform plan - preview changes

terraform apply - make it real

terraform destroy - tear it down

State

terraform state list

terraform state show <resource>

terraform state rm <resource>

terraform import <addr> <id>

Config Blocks

provider "aws" { region = ... }

resource "type" "name" { ... }

variable "x" { default = ... }

output "y" { value = ... }

Modules

module "name" { source = ... }

terraform get - fetch modules

count / for_each - repeat resources

Team Safety

Remote backend (S3, Terraform Cloud)

State locking (DynamoDB)

terraform fmt / validate before commit